



Tarea 09: Documentación de la Calidad

Informe Técnico Integrado del Proyecto Outline

ASIGNATURA

Estándares y Métricas para el Desarrollo de Software

DOCENTE

Dr(c). Raul Alejandro Velasquez Ortiz

PROYECTO INTEGRADOR

Outline - Wiki interna estilo Notion (Open Source maduro)

EQUIPO No. 4

CHAIRES ALBA OLIVIA LIZBETH	UP240594
ESTRADA RAMÍREZ NOÉ EZEQUIEL	UP240770
FACIO PALOS OMAR	UP240470
JAUREGUI DE LA RIVA ALAN RICARDO	UP240960
RUBIO VIRAMONTES BRIAN ODIN	UP250004

GRUPO

TIID05B

REPOSITORIO

<https://github.com/UP240594/outline-sqa-docs>

Aguascalientes, México
22 de julio de 2026

Índice

1. Introducción	2
1.1. Contexto del proyecto	2
1.2. Propósito de la investigación	2
1.3. Objetivos específicos	2
1.4. Estructura del documento	3
2. Investigación y anatomía de estándares	3
2.1. Comparativa de estándares de calidad y pruebas	3
2.2. Anatomía de los informes de calidad y pruebas	5
3. Matriz de trazabilidad y ejemplos de informes QA	7
3.1. Ejemplo de Matriz de Trazabilidad y Fundamentación de Auditoría	7
3.1.1. Ejemplo de Matriz de Trazabilidad: Historia de Usuario «Crear Documento»	7
3.2. Comparativa: Reporte de SonarQube vs. Reporte de Selenium	8
3.2.1. Tabla Comparativa de Información Reportada	8
3.2.2. Buenas Prácticas de Reporte Identificadas en Ambos Enfoques	8
4. Estrategia y Métricas de Calidad de Software	9
4.1. Investigación: métricas reportables e interpretación	9
4.1.1. Análisis e Interpretación de Métricas de Código y Pruebas	10
4.1.2. Cuadro de KPIs Definidos para el Proyecto Outline	11
5. Aplicación al proyecto Outline	11
5.1. Índice comentado del reporte	11
5.2. Introducción	12
5.3. Alcance	12
5.4. Metodología	12
5.5. Resultados: métricas del Examen Práctico U2	13
6. Conclusión integral	14
7. Aportes individuales	15

Introducción

Contexto del proyecto

El presente documento corresponde a la Tarea 09 de la asignatura Estándares y Métricas para el Desarrollo de Software, en la cual se investigan los estándares de documentación de la calidad y la estructura de los informes técnicos de calidad y de pruebas, aplicando esos hallazgos al reporte técnico integrado del proyecto integrador del equipo: **Outline**, una aplicación web de tipo wiki interna con funcionalidades similares a Notion, de código abierto y con una comunidad activa.

Outline fue seleccionado del Catálogo de 30 alternativas (Semana 1) por su naturaleza madura (más de 5,000 estrellas en GitHub, licencia MIT) y por ser un caso realista para evaluar atributos de calidad como usabilidad, seguridad y fiabilidad. El proyecto será analizado desde la perspectiva de un equipo de desarrollo que busca adoptar, desplegar y posiblemente contribuir a este software en un entorno corporativo.

Propósito de la investigación

Esta tarea tiene como propósito que el equipo investigue qué dicen los estándares y las fuentes recientes sobre cómo documentar la calidad y las pruebas del software, y que aplique esos hallazgos estructurando el reporte técnico integrado de su proyecto (entregable final de la Unidad III). Este documento constituye el primer borrador del reporte integrador: la investigación fundamenta la estructura, y la aplicación produce las primeras secciones (introducción, alcance y metodología).

Objetivos específicos

1. Comparar los estándares de documentación de calidad y pruebas: IEEE 730, ISO/IEC/IEEE 29119 e IEEE 829.
2. Investigar la anatomía del informe técnico de calidad y del informe de pruebas (secciones obligatorias).
3. Investigar el concepto de trazabilidad (requisito $\rightarrow$ caso $\rightarrow$ defecto $\rightarrow$ métrica) y su matriz.
4. Investigar qué métricas son reportables y cómo se interpretan (ISO/IEC 25010:2023 / 25023).
5. Analizar y comparar dos ejemplos reales de informes de aseguramiento de calidad.

6. Proponer la estructura del reporte integrador y redactar sus primeras secciones.

Estructura del documento

El documento se organiza en seis secciones principales de desarrollo (4.1 a 4.6): estándares de documentación, anatomía del informe técnico, trazabilidad, métricas reportables, ejemplos reales y aplicación al proyecto Outline. Finaliza con conclusión, aportes individuales y referencias.

Investigación y anatomía de estándares

Comparativa de estándares de calidad y pruebas

La adopción de estándares formalizados permite establecer procesos sistemáticos para el aseguramiento de la calidad y la ejecución de pruebas, proporcionando un marco de referencia que facilita la trazabilidad, la repetibilidad y la mejora continua. Para el proyecto *Outline* —una wiki interna de tipo Notion— se han analizado tres estándares internacionales representativos de distintas facetas de la calidad del software: IEEE 730-2014 (enfocado en el aseguramiento de la calidad, SQA), ISO/IEC/IEEE 29119-3 (que abarca el proceso completo de pruebas) e IEEE 829 (especializado en la documentación de pruebas). La Tabla 1 sintetiza sus características fundamentales.

Tabla 1: Comparativa de estándares para *Outline*

Criterio	IEEE 730-2014	ISO/IEC/IEEE 29119-3	IEEE 829
Propósito	Definir los procesos de aseguramiento de la calidad del software (SQA) para garantizar que el producto cumple los requisitos y estándares establecidos.	Especificar la documentación de pruebas, incluyendo los artefactos que deben generarse en cada nivel de prueba (unidad, integración, sistema, aceptación).	Establecer un proceso de pruebas unificado que abarca la gestión, el diseño, la ejecución y la evaluación de resultados, con énfasis en la organización sistemática de las actividades de prueba.
Documentos que define	Plan SQA, informes de auditoría, registros de no conformidades, informes de métricas de calidad.	Plan de pruebas, especificación de casos de prueba, informe de incidencias, informe de resultados de pruebas, matriz de trazabilidad.	Plan maestro de pruebas, especificaciones de diseño de pruebas, procedimientos de prueba, informes de ejecución y resúmenes de pruebas.
Vigencia y aplicabilidad	Aplicable a cualquier proyecto donde se requiera un enfoque formal de garantía de calidad; vigente desde 2014 y ampliamente utilizado en la industria.	Parte de la familia 29119, actualizada en 2021; aplicable a organizaciones que buscan estandarizar su proceso de pruebas independientemente del ciclo de vida.	Estándar histórico (1998) aún referenciado; ha sido reemplazado en parte por la norma 29119, pero su estructura de documentación sigue siendo una base práctica.

A partir del análisis comparativo, se concluye que el estándar más adecuado para el reporte de calidad de *Outline* es el **IEEE 730-2014**. Esta elección se justifica por tres razones principales:

1. **Alcance manejable:** *Outline* es un proyecto de equipo estudiantil con una base de código modular y un equipo reducido. IEEE 730 se enfoca en SQA, lo que permite establecer un plan de calidad sin la sobrecarga documental que implicaría adoptar la totalidad de ISO 29119, que es más exhaustivo y está orientado a organizaciones con procesos de pruebas maduros y equipos grandes.
2. **Énfasis en la trazabilidad y métricas:** El estándar IEEE 730 exige la definición de métricas y la documentación de no conformidades, lo que se alinea directamente con los entregables de este reporte, donde se requiere mostrar la trazabilidad entre requisitos, casos de prueba y defectos, así como métricas de cobertura y complejidad.
3. **Complementariedad práctica:** Aunque IEEE 829 proporciona una estructura detallada para la documentación de pruebas, su enfoque es más técnico y está orientado a la fase de pruebas, mientras que el enfoque gerencial de IEEE 730 permite integrar tanto los resultados de pruebas (Selenium) como el análisis estático (SonarQube) en un marco de aseguramiento de calidad global, tal como se ha realizado en el Examen Práctico de la Unidad 2.

Anatomía de los informes de calidad y pruebas

La documentación de la calidad en un proyecto de software requiere distinguir entre dos perspectivas complementarias: la **gerencial o de aseguramiento de la calidad (SQA)**, que se enfoca en la supervisión, los riesgos y el estado general del proyecto, y la **técnica o de pruebas**, que detalla la ejecución, los resultados y los defectos encontrados. A continuación se listan las secciones típicas de cada tipo de informe, basadas en la literatura de referencia y en la práctica documentada para *Outline* (Galin, 2018; IEEE, 2014).

Informe de Aseguramiento de la Calidad (SQA) – Enfoque gerencial

Este informe está dirigido a la gestión del proyecto y a los *stakeholders*, y resume el estado de la calidad desde una perspectiva de alto nivel. Las secciones habituales son:

- **Introducción:** Describe el propósito del informe, el alcance del proyecto y el contexto en el que se aplica el plan SQA.
- **Plan de calidad:** Incluye los objetivos de calidad, los estándares aplicados (ej. IEEE 730, ISO 25010), y la estrategia general para garantizar la calidad.
- **Gestión de riesgos:** Identifica los riesgos técnicos y organizacionales que afectan a la calidad, junto con las medidas de mitigación adoptadas.

- **Revisión y auditoría:** Resume las actividades de revisión de código, inspecciones y auditorías realizadas durante el desarrollo.
- **Métricas de calidad:** Presenta indicadores clave (KPIs) como cobertura de pruebas, complejidad ciclomática, deuda técnica, y su evolución en el tiempo.
- **No conformidades y acciones correctivas:** Documenta los defectos o desviaciones encontradas, su severidad y el plan de remediación.
- **Conclusiones y recomendaciones:** Evalúa el cumplimiento de los objetivos de calidad y propone mejoras para iteraciones futuras.

Informe de resultados de pruebas – Enfoque técnico

Este informe se centra en la ejecución de las pruebas y está dirigido al equipo de desarrollo y aseguramiento de calidad. Su propósito es proporcionar evidencia detallada del comportamiento del sistema. Las secciones típicas son:

- **Resumen ejecutivo de pruebas:** Visión general de los resultados, incluyendo el número total de pruebas ejecutadas, pasadas y fallidas, y el estado del *Quality Gate*.
- **Estrategia de pruebas:** Describe el enfoque de pruebas (funcional, no funcional, automatizada, manual), los niveles de prueba y los criterios de entrada/salida.
- **Diseño de casos de prueba:** Lista los casos de prueba agrupados por módulo o historia de usuario, especificando entradas, condiciones previas y resultados esperados.
- **Resultados de ejecución:** Detalla la ejecución de cada caso de prueba, con trazabilidad a los requisitos y evidencias (capturas de pantalla, *logs*).
- **Defectos encontrados:** Clasifica los defectos por severidad, estado y módulo afectado, incluyendo su ciclo de vida (abierto, en progreso, resuelto, cerrado).
- **Cobertura de pruebas:** Presenta métricas de cobertura de código, ramas y líneas, junto con las herramientas utilizadas (ej. *pytest-cov*, *Jest*).
- **Conclusiones y riesgos residuales:** Evalúa la calidad del sistema con base en los resultados y señala áreas que requieren atención adicional.

La distinción entre ambos enfoques es crucial porque el informe SQA permite a la dirección tomar decisiones estratégicas sobre la viabilidad del proyecto y la asignación de recursos, mientras que el informe de pruebas proporciona al equipo técnico la información granular necesaria para corregir defectos y mejorar la calidad del código (Galin, 2018; Pressman & Maxim, 2020).

Matriz de trazabilidad y ejemplos de informes QA

Ejemplo de Matriz de Trazabilidad y Fundamentación de Auditoría

En el ciclo de vida de pruebas del software, la matriz de trazabilidad garantiza que todo requerimiento del negocio posea su cobertura de prueba correspondiente, rastreando su ciclo de vida hasta las métricas operacionales de calidad.

Ejemplo de Matriz de Trazabilidad: Historia de Usuario «Crear Documento»

Requisito (Historia de Usuario): HU-DOC-001 — *Como usuario autenticado de la Wiki, quiero crear un nuevo documento en una colección para documentar procesos del equipo.*

Caso de Prueba (Test Case): TC-DOC-001-VAL — *Validar que al presionar el botón «+ New doc» en la barra lateral e ingresar un título válido, el documento se persista y redireccione a la ruta /doc/...*

Defecto (Bug Report): BUG-DOC-104 — *En el entorno de integración, el clic en la confirmación de la creación genera una excepción de timeouts por latencia en el orquestador local antes de renderizar.*

Métricas de Calidad:

- Porcentaje de ejecuciones pasadas (100 % esperado tras re-ejecución).
- Tiempo de respuesta de creación < 500 ms.
- Cobertura de código en `document_page.py` $\geq 80\%$ (alcanzado 95 % según evidencias del proyecto).

¿Por qué una Auditoría de Calidad exige esta Trazabilidad?

Desde el marco de estándares como **ISO/IEC 25010** y **CMMI Nivel 3** (Procesos Definidos), una auditoría de *Software Quality Assurance* (SQA) requiere evidencia matemática e irrefutable de la conformidad del producto. Las razones clave incluyen:

1. **Garantía de Cobertura de Requisitos (Sin Brechas):** Previene la entrega de funcionalidades no probadas (vacíos de prueba) o la construcción de código «fantasma» que no responde a ninguna necesidad del cliente.
2. **Análisis de Impacto:** Permite determinar exactamente qué pruebas y componentes se ven afectados cuando cambia una especificación o cuando falla un test funcional.
3. **Control de Deuda Técnica y Gestión de Defectos:** Relacionar métricas de cobertura con defectos específicos ayuda a justificar auditorías de remediación bajo la filosofía *Clean as You Code*.
4. **Validación Transparente y Auditable:** Ante evaluaciones externas o normativas formales (como **IEEE 730**), la trazabilidad bi-direccional demuestra que los criterios de aceptación fueron validados mediante métricas cuantitativas recopiladas en pipelines automáticos de CI/CD.

Comparativa: Reporte de SonarQube vs. Reporte de Selenium

En los procesos SQA de la plataforma Wiki *Outline*, se combinan dos técnicas de evaluación: **Análisis Estático** (SonarQube/SonarCloud) y **Análisis Dinámico Funcional** (Selenium WebDriver con Pytest).

Tabla Comparativa de Información Reportada

Buenas Prácticas de Reporte Identificadas en Ambos Enfoques

1. **Uso de Formatos Estándar e Intercambiables (XML / HTML):**
 - *Selenium/Pytest:* Genera reportes interactivos autocontenidos (`reporte.html`) y reportes estructurados de cobertura (`coverage.xml`).
 - *SonarQube:* Parsea y procesa directamente el artefacto `coverage.xml` para consolidar métricas dinámicas y estáticas en un único tablero central.
2. **Establecimiento de Criterios de Aprobación Explícitos (*Quality Gates*):** Definir un estatus global claro (*Passed / Failed*) que evalúe si el código cumple con los umbrales

Criterio	Reporte de SonarQube (Análisis Estático)	Reporte de Selenium / Pytest (Análisis Dinámico)
Dimensión Evaluada	Calidad Interna: Confiabilidad, Seguridad, Mantenibilidad, Cobertura y Duplicaciones.	Comportamiento Funcional: Interacción E2E, flujos de negocio e interfaz de usuario (UI/DOM).
Aporte de Información	■ Identificación de <i>Code Smells</i> y ternarios anidados. ■ Detección de <i>Security Hotspots</i>. ■ Deuda técnica calculada en días (23 días). ■ Porcentaje de código duplicado.	■ Estatus de ejecución (<i>Passed/Failed/Skipped</i>). ■ Excepciones en tiempo de ejecución (timeouts, aserciones). ■ Desempeño y tiempos de ejecución (42 passed en 125,22 s).
Fase del Ciclo de Vida	Inspecciona el código fuente sin ejecutar el programa (<i>Shift-left</i>).	Evalúa la aplicación en tiempo de ejecución con navegadores reales/headless.

mínimos de mantenibilidad, porcentaje de fallas y cobertura de código antes de autorizar una fusión (*merge*) en el repositorio.

3. Estructura y Desacoplamiento de la Información:

- *Diseño en Selenium:* Implementación del patrón *Page Object Model* (POM) para que los fallos del reporte mapeen directamente hacia clases de negocio (`login_page.py`, `search_page.py`).
- *Clasificación en SonarQube:* Categorización de los hallazgos por nivel de severidad (*Blocker*, *Critical*, *Major*, *Minor*) y asignación de responsables para la planificación de *sprints* de refactorización.

4. **Automatización e Integración Continua (CI/CD):** Ambas herramientas ejecutan sus reportes de forma automatizada en pipelines (GitHub Actions) en cada evento de *push* o *pull request*, asegurando la visibilidad del estado de la calidad para todo el equipo sin depender de ejecuciones manuales aisladas.

Estrategia y Métricas de Calidad de Software

Investigación: métricas reportables e interpretación

La evaluación cuantitativa del software requiere la definición formal de métricas tanto de producto (código fuente y estructura) como de proceso de pruebas (Galin, 2018; ISO/IEC, 2023). A continuación, se clasifican las métricas clave, sus umbrales de referencia y su corres-

pondencia con el modelo de calidad ISO/IEC 25010:2023 e ISO/IEC 5055:2021 (ISO/IEC, 2016, 2021).

Tabla 2: Métricas reportables de código y pruebas ligadas a ISO/IEC 25010:2023.

Categoría	Métrica	Fórmula / Indicador	Umbral de Referencia	Característica ISO 25010
Código	Complejidad Ciclomática $V(G)$	$V(G) = P + 1$ (McCabe, 1976)	$V(G) \leq 10$ por función	Mantenibilidad (Analizabilidad)
Código	Halstead (Volumen V)	$V = (N_1 + N_2) \log_2(n_1 + n_2)$	$V \leq 1000$ bits/módulo	Mantenibilidad (Comprensibilidad)
Código	Puntos de Función (IFPUG)	Conteo funcional de ILF, EIF, EI, EO, EQ	Según la historia (≤ 5 FP/HU)	Adecuación Funcional
Estructural	Reglas ISO/IEC 5055	Violaciones estáticas en 4 pilares	0 fallas críticas	Seguridad y Mantenibilidad
Pruebas	Cobertura de Código	$\frac{\text{Líneas Ejecutadas}}{\text{Líneas Totales}} \times 100$	$\geq 80\%$ (Outline: 62 %)	Testabilidad (Mantenibilidad)
Pruebas	Densidad de Defectos	$\frac{\text{Defectos Conf.}}{\text{KLOC}}$	$\leq 1,0$ defectos/KLOC	Confiabilidad (Madurez)
Pruebas	Tasa de Aprobación	$\frac{\text{Casos Aprobados}}{\text{Casos Ejecutados}} \times 100$	$\geq 95\%$ (Outline: 100 %)	Correctitud Funcional

Análisis e Interpretación de Métricas de Código y Pruebas

- **Complejidad Ciclomática de McCabe ($V(G)$):** Determina el número de caminos independientes en el flujo de ejecución (McCabe, 1976). Valores superiores a 10 reflejan baja mantenibilidad y exigen división modular.
- **Métricas de Halstead:** Evalúan la complejidad cuantitativa midiendo operadores (n_1, N_1) y operandos (n_2, N_2) únicos y totales (Galin, 2018). Un volumen o dificultad elevado incrementa la probabilidad de inconsistencias lógicas.
- **Métricas Estructurales ISO/IEC 5055:2021:** Analizan cuantitativamente las vulnerabilidades en código fuente que impactan en la confiabilidad y mantenibilidad, detectando patrones indeseables antes del despliegue (ISO/IEC, 2021).
- **Métricas de Pruebas (Cobertura y Densidad):** La cobertura de código permite verificar la efectividad de los casos de prueba ejecutados (e.g. Selenium/PyTest). La densidad de defectos valida la estabilidad de las funcionalidades del sistema (ISTQB, 2024).

Cuadro de KPIs Definidos para el Proyecto Outline

A partir de la especificación de calidad de la wiki interna *Outline*, se consolidan los 7 KPIs clave de desempeño y calidad de software (Pressman & Maxim, 2020):

1. **KPI-01 (Complejidad Ciclomática):** $V(G) \leq 8$ promedio por módulo de edición.
2. **KPI-02 (Duplicación de Código):** Duplicación $< 3\%$ según reporte estático SonarQube / ISO 5055.
3. **KPI-03 (Cobertura de Pruebas):** Cobertura de código automatizado $\geq 80\%$ (Meta de refactorización respecto al 62% de P2).
4. **KPI-04 (Densidad de Defectos):** $< 0,8$ defectos por KLOC en etapa de integración.
5. **KPI-05 (Tasa de Aprobación de Pruebas):** 100% de aprobación en la suite de regresión funcional (42/42 pruebas pasadas).
6. **KPI-06 (Seguridad en Código):** 0 vulnerabilidades de severidad Alta/Bloqueante.
7. **KPI-07 (Comportamiento Temporal):** Tiempo de respuesta < 500 ms al renderizar notas Markdown.

Aplicación al proyecto Outline

Índice comentado del reporte

A modo de esqueleto, el reporte de aplicación se organiza en los siguientes apartados, cada uno con el contenido que le corresponde:

- **Introducción:** presenta el objetivo del reporte y describe brevemente qué es Outline y por qué se documenta su calidad.
- **Alcance:** delimita los módulos de Outline sometidos a prueba y aquellos que quedaron fuera del análisis.
- **Metodología:** explica el enfoque empleado en el Examen Práctico de la Unidad 2 (pruebas funcionales con Selenium y análisis estático con SonarQube).
- **Resultados:** consolida las métricas cuantitativas obtenidas (cobertura, deuda técnica, casos de prueba y *Quality Gate*).
- **Conclusiones:** sintetiza los hallazgos clave y las recomendaciones de remediación de la deuda técnica detectada.

Introducción

El presente apartado documenta la aplicación práctica de las estrategias de aseguramiento de la calidad sobre el proyecto integrador **Outline**, una plataforma *open source* de gestión del conocimiento empresarial de arquitectura similar a Notion. A diferencia de las secciones anteriores —de carácter conceptual—, aquí se reportan resultados concretos: la forma en que se auditó la calidad del sistema mediante pruebas funcionales automatizadas y análisis estático de código, así como las métricas obtenidas de dicho ejercicio.

Los resultados se enmarcan en el modelo de calidad de producto **ISO/IEC 25010:2023** —que define características como la mantenibilidad, la fiabilidad y la eficiencia de desempeño— y en el estándar **IEEE 730**, que estructura el plan de aseguramiento de la calidad del software (SQA). El objetivo es evidenciar, de manera trazable y verificable, el estado de calidad del módulo evaluado de Outline.

Alcance

El análisis práctico se concentró en los flujos funcionales críticos de Outline accesibles desde su interfaz web, ejecutados sobre una instancia desplegada localmente:

- **Autenticación:** ingreso mediante correo electrónico y el flujo de inicio de sesión.
- **Búsqueda de documentos:** recuperación de artículos existentes y manejo de resultados vacíos.
- **Creación y edición documental:** validación del flujo de alta de documentos dentro del *wiki*.

Estos módulos se modelaron mediante el patrón Page Object Model (POM), con clases dedicadas (`base_page`, `login_page`, `search_page` y `document_page`).

Quedaron **fuera del alcance** de las pruebas automatizadas: la colaboración en tiempo real, las integraciones con servicios externos (por ejemplo, autenticación real vía Slack o *webhooks*), la totalidad del *backend* y el renderizado recursivo del árbol de navegación de colecciones (analizado de forma teórica en la Unidad 1, pero no cubierto por la suite E2E).

Metodología

El aseguramiento de la calidad se abordó con un enfoque dual y complementario:

- **Análisis dinámico (pruebas funcionales E2E):** se implementó una suite de pruebas de extremo a extremo con **Selenium WebDriver** y **pytest**, bajo el patrón Pa-

ge Object Model. La sincronización se resolvió exclusivamente con esperas explícitas (`WebDriverWait`), evitando pausas fijas, y se incorporaron pruebas basadas en datos (*data-driven*) mediante el decorador `@pytest.mark.parametrize`. La cobertura se midió con `pytest-cov`, exportando los reportes `coverage.xml` y `report.html`.

- **Análisis estático:** se auditó el código fuente con **SonarQube v10.6**, evaluando sus cinco dimensiones (Seguridad, Confiabilidad, Mantenibilidad, Cobertura y Duplicación) y aplicando un *Quality Gate* como criterio automático de aprobación.

El entorno de ejecución se levantó localmente mediante Docker (contenedores de Redis y PostgreSQL) sobre el dominio `local.outline.dev:3000`, y el flujo se automatizó con un *pipeline* de Integración Continua en GitHub Actions.

Nota: conforme a la naturaleza documental de esta tarea, esta sección **no ejecuta nuevas pruebas**; consolida y documenta los resultados previamente obtenidos en el Examen Práctico de la Unidad 2 (P2).

Resultados: métricas del Examen Práctico U2

La Tabla 3 sintetiza las métricas cuantitativas obtenidas durante el Examen Práctico de la Unidad 2, junto con su interpretación.

Tabla 3: Métricas de calidad obtenidas en el Examen Práctico de la Unidad 2 (P2).

Métrica	Valor	Interpretación
Cobertura de código	62 %	Porcentaje de líneas del paquete de pruebas ejercitadas por la suite, reportado por <code>pytest-cov</code> . Se ubica por debajo del umbral ideal (>80 %), lo que señala rutas de código aún sin verificar. Se vincula con la Mantenibilidad de ISO/IEC 25010.
Deuda técnica	23 días	Esfuerzo estimado por SonarQube para remediar la totalidad de los <i>code smells</i> detectados; equivale a jornadas de refactorización pendientes.
Pruebas pasadas	42/42	Todos los casos ejecutados (42 de 42) resultaron exitosos (<i>passed</i>), validando funcionalmente los flujos de autenticación, búsqueda y creación documental; adicionalmente, 8 casos quedaron omitidos (<i>skipped</i>) por dependencias del entorno.
Quality Gate	Aprobado	El proyecto satisfizo los umbrales del <i>Quality Gate</i> definido en SonarQube; la puerta de calidad no bloqueó la integración del código.

En conjunto, las métricas indican un sistema funcionalmente estable —la totalidad de los casos ejecutados pasaron— que satisface el *Quality Gate* configurado. No obstante, es importante señalar que las dimensiones de **Seguridad** y **Confiabilidad** obtuvieron una calificación *rating* D, lo que revela una deuda técnica considerable (23 días) y la necesidad de priorizar refactorizaciones en los *sprints* venideros para aproximar dichas dimensiones al estándar ideal A.

Conclusión integral

La aplicación de los estándares ISO/IEC 25010, IEEE 730, CMMI y MoProSoft al proyecto Outline ha permitido al equipo descubrir varios aprendizajes clave:

Hallazgo 1: La priorización es esencial. No todas las características de calidad tienen el mismo peso. Para una wiki interna, usabilidad y seguridad dominan sobre portabilidad.

Hallazgo 2: Los KPIs deben ser realistas. Definir umbrales como disponibilidad 99,9 % es alcanzable; 99,999 % no lo es para un proyecto open-source.

Hallazgo 3: IEEE 730 es estructurado pero adaptable. Adaptarlo a un equipo de 5

personas requirió recortar secciones no aplicables.

Hallazgo 4: MoProSoft es relevante y vigente. Sigue siendo pertinente para PYMEs locales, con principios de baja burocracia.

Hallazgo 5: La trazabilidad es clave para auditorías. La matriz de trazabilidad requisito $\rightarrow$ caso de prueba $\rightarrow$ defecto $\rightarrow$ métrica permite demostrar que cada funcionalidad ha sido probada y que los defectos se gestionan adecuadamente.

Conexión con próximos parciales: Los KPIs definidos serán la base para las pruebas del Parcial 2, y la matriz de riesgos alimentará el informe técnico del Parcial 3.

Este trabajo ha permitido al equipo comprender la importancia de documentar la calidad de manera sistemática, aplicando estándares reconocidos internacionalmente y adaptándolos al contexto específico de un proyecto de código abierto como Outline.

Aportes individuales

Chairez Alba Olivia Lizbeth (UP240594) — Coordinadora

Redacté la portada, introducción, conclusión y la sección de referencias unificadas. Unifiqué y armonicé todas las partes de mis compañeros en un solo documento LaTeX, corregí errores de compilación y formateé el documento final. Gestioné el repositorio Git y aseguré la consistencia general. Además, consolidé la estructura del reporte integrador y verifiqué que todas las citas y referencias cumplieran con el formato APA 7.

Estrada Ramírez Noé Ezequiel (UP240770) — Líder / Analista

Durante el desarrollo de esta investigación, coordiné la estructuración analítica de las métricas de calidad de software aplicables al proyecto Outline, integrando las especificaciones de ISO/IEC 25010:2023, ISO/IEC 25023 e ISO/IEC 5055:2021. Fundamenté la relación entre las métricas de código (McCabe, Halstead e IFPUG) y las métricas de pruebas automáticas (cobertura y densidad de defectos), definiendo los umbrales de tolerancia técnica y consolidando el conjunto de KPIs del proyecto. Este análisis permitió establecer criterios claros para interpretar los resultados del análisis estático y dinámico en el reporte integrador.

Facio Palos Omar (UP240470) — QA Tester

Elaboré las secciones de trazabilidad (4.3) y ejemplos reales de QA (4.5), definiendo la matriz de trazabilidad con un ejemplo concreto basado en la historia de usuario HU-01 (Crear docu-

mento) de Outline. Además, realicé una comparativa detallada entre los reportes generados por SonarQube (análisis estático) y Selenium (análisis dinámico), identificando las buenas prácticas de reporte y las carencias de cada enfoque.

Jauregui De La Riva Alan Ricardo (UP240960) — Documentador

Redacté las secciones 4.1 (estándares de documentación) y 4.2 (anatomía del informe técnico), comparando los estándares IEEE 730-2014, ISO/IEC/IEEE 29119-3 e IEEE 829, y justificando la elección de IEEE 730 para el proyecto Outline. Además, describí las secciones obligatorias de los informes de calidad (SQA) y de pruebas, distinguiendo el enfoque gerencial del técnico.

Rubio Viramontes Brian Odin (UP250004) — Developer

Desarrollé la sección 4.6 (Aplicación), que incluye el índice comentado del reporte integrador, la redacción de la introducción, alcance y metodología del proyecto Outline, y la consolidación de las métricas obtenidas en el Examen Práctico de la Unidad 2 (P2), incluyendo la cobertura de código (62 %), la deuda técnica (23 días), las pruebas pasadas (42/42) y el estado del Quality Gate (Aprobado).

Referencias

- Galín, D. (2018). *Software Quality: Concepts and Practice*. Wiley-IEEE Computer Society.
- IEEE. (2014). *IEEE Standard for Software Quality Assurance Processes (IEEE Std 730-2014)*. <https://doi.org/10.1109/IEEESTD.2014.6835311>
- ISO/IEC. (2016). *ISO/IEC 25023:2016 — Systems and software engineering — SQuaRE — Measurement of product quality*. ISO/IEC.
- ISO/IEC. (2021). *ISO/IEC 5055:2021 — Automated Source Code Quality Measures*. ISO/IEC.
- ISO/IEC. (2023). *ISO/IEC 25010:2023 — Systems and software engineering — SQuaRE — Product quality model*. ISO/IEC.
- ISTQB. (2024). *ISTQB Glossary of Testing Terms v4.4*. <https://glossary.istqb.org>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308-320. <https://doi.org/10.1109/TSE.1976.233837>
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th). McGraw-Hill.